Technical Report — CPU Scheduling Simulator

Project: `project_323.1`
Author: DewanMim
Language: Java 11 + Swing
Build Tool: Maven

1. Executive Summary

The CPU Scheduling Simulator is a Java Swing-based desktop application designed to evaluate and visualize classic operating system scheduling algorithms, including First-Come First-Served (FCFS), Shortest Job Next (SJN), and Round Robin (RR). During the development of the simulation engine and the accompanying graphical user interface (GUI), several intricate architectural and algorithmic challenges arose. This report outlines four major challenges encountered, detailing the situation, task, action, and resulting outcome (STAR format) for each.

---

2. Challenge 1 — SJN Idle-Time Gap Management

Situation

When implementing the **Shortest Job Next (SJN)** algorithm, the scheduler iteratively selects the process with the smallest burst time from the pool of arrived processes. A critical edge case emerges when the CPU completes a process, but the next process has not yet arrived. This creates a "CPU idle period." A naive looping approach would result in an infinite loop or select an un-arrived process, corrupting the waiting times.

Task

The objective was to implement robust SJN logic that accurately handles idle gaps by advancing the simulation clock to the exact arrival time of the next available process, without compromising the algorithm's time complexity.

Action

A stream-based candidate filtering mechanism was introduced. The algorithm filters candidates based on their arrival time relative to the current simulation time. If no candidate is found (indicating an idle period), the simulation clock gracefully jumps directly to the arrival time of the next upcoming process:

```
Process shortest = remaining.stream()
    .filter(p -> p.getArrivalTime() <= finalCurrentTime)
    .min(Comparator.comparingInt(Process::getBurstTime))
    .orElse(null);

if (shortest == null) {
    // Fast-forward simulation clock to the next process arrival
    int nextArrival = remaining.stream()
            .mapToInt(Process::getArrivalTime)
            .min().orElse(currentTime + 1);
    currentTime = nextArrival;
}
```

Result

The SJN scheduler now inherently supports sparse arrival data sets. The simulation skips over idle gaps in O(1) logical steps per gap, keeping performance linear relative to process count. The Gantt chart accurately omits drawing blocks during idle times.

---

3. Challenge 2 — Round Robin Ready-Queue Ordering

Situation

The **Round Robin (RR)** algorithm's accuracy is highly sensitive to the order in which processes are enqueued into the ready queue. The correct operating system convention dictates that newly-arrived processes must be added to the queue *before* the currently preempted process is re-queued. Failure to enforce this order results in incorrect execution sequences and inflated turnaround times.

Task

Implement a preemptive scheduler with an explicit queue structure that handles intra-quantum arrivals correctly, and seeds the queue appropriately at simulation start (`currentTime = 0`) or after idle gaps.

Action

An `ArrayDeque` was utilized as a high-performance FIFO ready queue. A secondary pointer (`index`) was used to lazily track arrivals from a pre-sorted process list. The logic strictly dictates that arriving processes are evaluated and enqueued first, immediately after the execution quantum finishes:

```
// 1. Enqueue newly arrived processes that arrived during the quantum
while (index < n && sortedProcesses.get(index).getArrivalTime() <= currentTime) {
    readyQueue.add(sortedProcesses.get(index));
    index++;
}

// 2. Re-enqueue the preempted process only after new arrivals
if (currentProcess.getRemainingTime() > 0) {
    readyQueue.add(currentProcess);
}
```

Result

The RR simulation reliably reflects standard OS behavior, properly interleaving context switches. The use of `ArrayDeque` guarantees O(1) queue operations, resulting in an efficient and accurate Gantt timeline even under tight time quantums.

---

4. Challenge 3 — Mutable State Isolation Across Runs

Situation

The simulation allows users to execute multiple scheduling algorithms sequentially without restarting the application. Because `Process` objects maintain execution state (e.g., `remainingTime` , `startTime`), running a second simulation on the same object references causes errors. For instance, a process

completing in the first run has `remainingTime = 0`, causing it to be entirely skipped in the subsequent run.

Task

Design a mechanism to reset or isolate process state between distinct simulation executions, ensuring that subsequent runs operate on pristine data without overwriting the user's input table state.

Action

A deep copy paradigm was introduced via a **copy constructor** within the `Process` data model. When the `Scheduler` accepts a new process, it clones the data attributes while explicitly resetting execution metrics:

```
public Process(Process other) {
    this.id = other.id;
    this.arrivalTime = other.arrivalTime;
    this.burstTime = other.burstTime;
    this.remainingTime = other.burstTime; // Reset for fresh run
    this.startTime = -1;                  // Reset state flags
}
```

Result

Consecutive simulations are entirely decoupled. Users can seamlessly switch algorithms or adjust time quantums without state pollution. This architectural decision also guarantees safety if background threading were introduced in the future.

---

5. Challenge 4 — Dynamic Gantt Chart Visualization

Situation

Rendering the Gantt chart using Java2D required fitting an unpredictable timeline length into a fixed-height, variable-width `JPanel`. Hardcoded scaling would either crop large datasets or under-utilize space for short ones. Furthermore, text labels (Process IDs) would overflow and clutter the UI if execution slices were too narrow.

Task

Develop a responsive rendering routine that dynamically calculates pixel scaling based on the total simulation duration and the current window dimensions, while employing smart label suppression to maintain visual clarity.

Action

The `drawGanttChart` method calculates a dynamic `scale` factor at runtime by dividing the available panel width by the maximum simulation time (`maxTime`). A string-width measurement guard was added to prevent label overflow:

```
int usableWidth = ganttChartPanel.getWidth() - 2 * marginX;
double scale = (double) usableWidth / maxTime;
```

```
// Determine width and draw rectangle
int currentWidth = (int) ((record.endTime - record.startTime) * scale);
g2d.fillRect(x, y, currentWidth, rectHeight);

// Guard against label overflow
FontMetrics fm = g2d.getFontMetrics();
if (currentWidth >= fm.stringWidth(record.processId)) {
    g2d.drawString(record.processId, centerX, centerY);
}
```

Result

The Gantt chart dynamically adapts to any window size and any sequence duration. The timeline always spans the full available width, providing a highly professional and robust user experience. Text labels neatly hide themselves when space is insufficient, preventing graphical artifacts.

Conclusion: Resolving these four challenges significantly elevated the stability, accuracy, and presentation quality of the CPU Scheduling Simulator, resulting in an application suitable for academic demonstration and study.